

Supplementary Materials

Energy-efficient codon optimization on thermodynamic hardware

Andraž Jelinčič,^{1,*} Ross C. Walker¹

¹Extropic Corp., Boston, MA

Supplementary Note 1: Domain-wall encoding derivation

This note provides the full mathematical derivation of how Potts model weights are compiled to Ising model weights via domain-wall encoding (DWE) Chancellor [2019], Berwald et al. [2022], as referenced in Section 4.4 of the main text.

Setup and notation

Consider a single Potts variable $c \in \{0, 1, \dots, K-1\}$ with energy $E(c) = h_c$ for each state. We seek an Ising model on $\{-1, +1\}$ spins that reproduces these energies exactly (up to a constant) on valid configurations and penalizes invalid ones.

Thermometer encoding and constraint

Introduce $K-1$ Ising spins $s_1, \dots, s_{K-1} \in \{-1, +1\}$. Potts state k is encoded as the “thermometer” pattern Chancellor [2019] where the first k spins are $+1$ and the rest are -1 :

$$c = k \iff s_i = \begin{cases} +1 & i \leq k \\ -1 & i > k \end{cases} \quad (\text{S1})$$

The “domain wall”—the boundary between the $+1$ and -1 regions—sits at position k .

Invalid configurations (those with a -1 followed by a $+1$, i.e. a reverse domain wall) are penalized. In $\{0, 1\}$ variables $q_i = (1 + s_i)/2$, the penalty for a single reverse wall between positions i and $i+1$ is $P \cdot q_{i+1}(1 - q_i)$. Converting to $\{-1, +1\}$ spins:

$$H_{\text{constr}} = \frac{P}{4} \sum_{i=1}^{K-2} (1 - s_i + s_{i+1} - s_i s_{i+1}) \quad (\text{S2})$$

This equals zero on all valid (thermometer) configurations and adds P for each reverse domain wall. The constraint graph is a *chain* ($K-2$ edges), which is 2-colorable—in contrast to the $K(K-1)/2$ -edge clique required by one-hot encoding Lucas [2014].

*Corresponding author: andraz@extropic.ai

Single-variable Ising Hamiltonian

The indicator for Potts state k can be written as $\delta_{c=k} = q_k - q_{k+1}$ (defining $q_0 \equiv 1$, $q_K \equiv 0$). Therefore $\sum_k h_k \delta_{c=k} = h_0 + \sum_{i=1}^{K-1} (h_i - h_{i-1}) q_i$. Converting to $\{-1, +1\}$ spins:

$$H_{\text{bias}} = \sum_{i=1}^{K-1} \frac{h_i - h_{i-1}}{2} s_i + \text{const} \quad (\text{S3})$$

Combining Eq. (S2) and Eq. (S3) and collecting terms:

$$H = \sum_{i=1}^{K-1} b_i s_i - \frac{P}{4} \sum_{i=1}^{K-2} s_i s_{i+1} + \text{const} \quad (\text{S4})$$

where the local fields are:

$$b_i = \frac{h_i - h_{i-1}}{2} + \begin{cases} -P/4 & i = 1 \\ 0 & 2 \leq i \leq K-2 \\ +P/4 & i = K-1 \end{cases} \quad (\text{S5})$$

The couplings $J_{i,i+1} = -P/4$ are **ferromagnetic** (favoring aligned spins), forming a nearest-neighbor chain.

Worked example: $K = 4$

Consider Potts biases h_0, h_1, h_2, h_3 . Three Ising spins s_1, s_2, s_3 yield the Hamiltonian:

$$H = b_1 s_1 + b_2 s_2 + b_3 s_3 - \frac{P}{4} s_1 s_2 - \frac{P}{4} s_2 s_3 + \text{const} \quad (\text{S6})$$

with

$$b_1 = \frac{h_1 - h_0}{2} - \frac{P}{4}, \quad b_2 = \frac{h_2 - h_1}{2}, \quad b_3 = \frac{h_3 - h_2}{2} + \frac{P}{4} \quad (\text{S7})$$

Multi-variable case: pairwise interactions

Consider L Potts variables $c_1, \dots, c_L$ with unary biases $h_k^{(p)}$ and pairwise interactions $V_{kl}^{(pq)}$ Chancellor [2019], Berwald et al. [2022]:

$$E = \sum_p \sum_k h_k^{(p)} \delta_{c_p=k} + \sum_{(p,q)} \sum_{k,l} V_{kl}^{(pq)} \delta_{c_p=k} \delta_{c_q=l} \quad (\text{S8})$$

Each position p gets its own domain-wall chain of $K_p - 1$ spins $s_{p,1}, \dots, s_{p,K_p-1}$. The unary terms convert exactly as before (Eqs. (S3) and (S5)). Below we derive how pairwise Potts interactions compile to Ising couplings.

Step 1: Abel summation in $\{0, 1\}$ variables. Using the indicator identity $\delta_{c_p=k} = q_{p,k} - q_{p,k+1}$ (with $q_{p,0} \equiv 1$, $q_{p,K_p} \equiv 0$) and the standard Abel summation formula $\sum_{k=0}^{K-1} f_k (q_k - q_{k+1}) = f_0 + \sum_{i=1}^{K-1} (f_i - f_{i-1}) q_i$, we expand the pairwise energy between positions p and q .

First, sum over l for fixed k :

$$\sum_l V_{kl} (q_{q,l} - q_{q,l+1}) = V_{k,0} + \sum_{j=1}^{K_q-1} (V_{k,j} - V_{k,j-1}) q_{q,j} \quad (\text{S9})$$

Then sum over k , applying Abel summation to each coefficient separately:

$$E_{pq} = V_{0,0} + \sum_{i=1}^{K_p-1} \alpha_i q_{p,i} + \sum_{j=1}^{K_q-1} \beta_j q_{q,j} + \sum_{i=1}^{K_p-1} \sum_{j=1}^{K_q-1} \tilde{V}_{ij} q_{p,i} q_{q,j} \quad (\text{S10})$$

where

$$\alpha_i = V_{i,0} - V_{i-1,0} \quad (\text{first difference along column } l=0) \quad (\text{S11})$$

$$\beta_j = V_{0,j} - V_{0,j-1} \quad (\text{first difference along row } k=0) \quad (\text{S12})$$

$$\tilde{V}_{ij} = V_{ij} - V_{i-1,j} - V_{i,j-1} + V_{i-1,j-1} \quad (\text{mixed second difference}) \quad (\text{S13})$$

Step 2: convert to $\{-1, +1\}$ spins. Substitute $q_{p,i} = (1 + s_{p,i})/2$ into Eq. (S10). The linear and quadratic terms expand as:

$$\alpha_i q_{p,i} = \frac{\alpha_i}{2} + \frac{\alpha_i}{2} s_{p,i} \quad (\text{S14})$$

$$\tilde{V}_{ij} q_{p,i} q_{q,j} = \frac{\tilde{V}_{ij}}{4} + \frac{\tilde{V}_{ij}}{4} s_{p,i} + \frac{\tilde{V}_{ij}}{4} s_{q,j} + \frac{\tilde{V}_{ij}}{4} s_{p,i} s_{q,j} \quad (\text{S15})$$

Collecting all terms by type yields:

Inter-position couplings.

$$J_{(p,i),(q,j)}^{\text{inter}} = \frac{\tilde{V}_{ij}}{4} = \frac{V_{ij} - V_{i-1,j} - V_{i,j-1} + V_{i-1,j-1}}{4} \quad (\text{S16})$$

Bias contributions from pairwise interactions. Each pairwise interaction (p, q) contributes an additional bias to spins at both positions. For spin $s_{p,i}$:

$$b_{p,i}^{(pq)} = \frac{\alpha_i}{2} + \frac{1}{4} \sum_{j=1}^{K_q-1} \tilde{V}_{ij} = \frac{(V_{i,0} - V_{i-1,0}) + (V_{i,K_q-1} - V_{i-1,K_q-1})}{4} \quad (\text{S17})$$

where the simplification uses the fact that $\sum_j \tilde{V}_{ij}$ telescopes. Symmetrically, for spin $s_{q,j}$:

$$b_{q,j}^{(pq)} = \frac{(V_{0,j} - V_{0,j-1}) + (V_{K_p-1,j} - V_{K_p-1,j-1})}{4} \quad (\text{S18})$$

Complete multi-position Ising Hamiltonian. Combining unary terms (Eq. (S5)), constraint chains (Eq. (S2)), and pairwise terms (Eqs. (S16) to (S18)):

$$H = \sum_p \sum_{i=1}^{K_p-1} b_{p,i} s_{p,i} - \frac{P}{4} \sum_p \sum_{i=1}^{K_p-2} s_{p,i} s_{p,i+1} + \sum_{(p,q)} \sum_{i,j} \frac{\tilde{V}_{ij}^{(pq)}}{4} s_{p,i} s_{q,j} + \text{const} \quad (\text{S19})$$

The total bias on spin $s_{p,i}$ is:

$$b_{p,i} = \underbrace{\frac{h_i^{(p)} - h_{i-1}^{(p)}}{2}}_{\text{Potts unary}} + \underbrace{\text{boundary}_P(i)}_{\text{constraint}} + \underbrace{\sum_{q: (p,q) \in \mathcal{E}} b_{p,i}^{(pq)}}_{\text{pairwise (Eq. (S17))}} \quad (\text{S20})$$

where $\text{boundary}_P(i) = -P/4$ if $i = 1$, $+P/4$ if $i = K_p - 1$, and 0 otherwise.

The Ising model has two types of edges:

- **Constraint chains** (coupling $-P/4$, ferromagnetic): $s_{p,i} \leftrightarrow s_{p,i+1}$ within each position.
- **Inter-position couplings** (coupling $\tilde{V}_{ij}/4$): $s_{p,i} \leftrightarrow s_{q,j}$ for each pair of interacting Potts positions, with $(K_p-1)(K_q-1)$ such couplings per Potts edge.

P-independent mixing at the domain wall

A Gibbs update at the domain wall boundary—where spin s_i has neighbor $s_{i-1} = +1$ (inside the wall) and $s_{i+1} = -1$ (outside)—sees an effective field:

$$h_{\text{eff}} = b_i + J \cdot s_{i-1} + J \cdot s_{i+1} = b_i + J \cdot (+1) + J \cdot (-1) = b_i \quad (\text{S21})$$

The two ferromagnetic coupling contributions cancel exactly. The domain wall therefore performs a random walk governed only by the bias landscape $\{b_i\}$, completely independent of the penalty strength P .

This is the key advantage of DWE over one-hot encoding. In one-hot encoding, the penalty clique creates genuine energy barriers that scale with P , leading to exponentially slow mixing at large P Berwald et al. [2022], Levin et al. [2009]. In DWE, the constraint can be made arbitrarily strong without affecting the sampling dynamics at the domain wall.

4-color block decomposition

The Ising graph for the codon problem is 4-colorable using the coloring $(p_{\text{parity}}, j_{\text{parity}})$, where $p_{\text{parity}} = p \bmod 2$ is the position parity and $j_{\text{parity}} = j \bmod 2$ is the spin-index parity within each position’s chain. The key observation is that constraint edges connect spins within the same position (same p_{parity} , different j_{parity}), while inter-position edges connect spins at adjacent positions (different p_{parity}). Therefore, no two spins sharing the same $(p_{\text{parity}}, j_{\text{parity}})$ pair are connected by an edge, and each of the four color classes can be updated simultaneously in a parallel Gibbs step.

Comparison of encoding schemes

Table S1 summarizes the key properties of three encoding schemes for mapping a K -state Potts variable to binary spins.

Table S1: Comparison of encoding schemes for a K -state Potts variable. Values in parentheses are for $K = 6$ (the maximum in the codon problem).

Property	Native Potts	One-hot	Domain wall	Binary
Variables	1 categorical	K (6)	$K-1$ (5)	$\lceil \log_2 K \rceil$ (3)
Constraint edges	0	$\frac{K(K-1)}{2}$ (15)	$K-2$ (4)	complex
Penalty strength	N/A	large	$\sim 2-5\times$ smaller	complex
Chromatic impact	none	$\geq K$ -fold	none (2-colorable)	unpredictable
Gibbs mixing	optimal	poor at large P	P -independent	very poor

Domain-wall encoding offers several advantages over one-hot encoding Chancellor [2019], Berwald et al. [2022], Chen et al. [2021]. The constraint graph is a chain rather than a clique, yielding a much sparser graph and lower penalty requirements. The chain constraint is 2-colorable, so it at most doubles the chromatic number of the overall graph—unlike one-hot, which can increase it K -fold.

DWE has been proven optimal in the number of binary variables for quadratic interactions Berwald et al. [2022]. Finally, Chen et al. [2021] demonstrated experimentally that DWE outperforms one-hot encoding, with a D-Wave 2000Q using DWE outperforming the next-generation D-Wave Advantage with one-hot encoding. Binary encoding uses the fewest variables ($\lceil \log_2 K \rceil$) but introduces complex, non-local constraint structures that are difficult to enforce and lead to poor mixing.

Supplementary Note 2: Energy estimation details

This note provides the detailed methodology for the energy estimates presented in Section 4.7 of the main text.

TSU energy model (Ising chip)

We use the energy model from Jelincic et al. [2025] (Appendix E), which provides a physical model of an all-transistor Boltzmann machine Gibbs sampler. The total energy for a sampling run is:

$$E = n_{\text{iter}} \times N \times E_{\text{cell}} \quad (\text{S22})$$

where n_{iter} is the number of Gibbs iterations, N is the number of spins, and $E_{\text{cell}} \approx 1.3$ fJ is the energy per spin per Gibbs step.

The per-cell energy E_{cell} breaks down into four components:

- $E_{\text{rng}} \approx 350$ aJ: energy consumed by the all-transistor random number generator, based on direct measurements from a prototype chip. The RNG exploits shot-noise dynamics of sub-threshold transistors and has a decorrelation time of ~ 100 ns.
- E_{bias} : energy consumed by the biasing circuitry (a linear resistor network that computes a biasing voltage from neighbor states).
- E_{clock} : energy consumed by clocking circuitry that orchestrates the sequential color-block updates.
- E_{comm} : energy consumed by inter-cell communication (charging wires to neighboring cells).

This model captures all central functional units of a hardware Boltzmann machine sampler. When comparing the results of this type of calculation to a detailed analysis of a complete device design (Cadence simulations), agreement is found within an order of magnitude Jelincic et al. [2025].

For the codon optimization problem, the Ising model has $N = 3,147$ spins with a maximum degree of 12 and only local connectivity, closely matching the setup analyzed in Jelincic et al. [2025]. With $n_{\text{iter}} = 2 \times 10^4$ Gibbs iterations:

$$E_{\text{Ising}} = 2 \times 10^4 \times 3,147 \times 1.3 \times 10^{-15} \text{ J} \approx 82 \text{ nJ} \quad (\text{S23})$$

TSU energy model (Potts chip, projected)

The Potts chip energy estimate is based on a physical model of a p-dit (categorical) sampling circuit, extrapolated from prototype measurements of p-bit circuits. The key energy-consuming components are:

- **P-bit sampling:** each p-cell draws ~ 800 aJ per random sample from its internal stochastic circuit.
- **MACDAC** (multiply-accumulate DAC): computes the conditional bias for each Potts state from neighbor states. For a node with q states and d neighbors, the MACDAC has $d \cdot \lceil \log_2 q \rceil + 2$ inputs, with an output capacitance of ~ 500 aF per input.
- **Communication:** wires carrying state information to neighboring cells, with capacitance ~ 350 aF/ μm .
- **Clocking:** global clock distribution for coordinating block updates.

Each p-cell requires $\sim q \times 10$ relaxation steps for its internal Markov chain (where q is the number of Potts states), yielding a total energy of ~ 0.084 nJ per Gibbs sweep for $L = 1,273$ Potts nodes. Over 10 sweeps: $10 \times 0.084 \approx 0.84$ nJ.

We note that Potts (p-dit) hardware does not yet exist; only the Ising (p-bit) TSU is the near-term target. For an example of how a Potts model could be implemented in CMOS hardware, see Whitehead et al. Whitehead et al. [2023].

GPU FLOP counting

Table S2 summarizes the FLOP estimates for each algorithm. These are order-of-magnitude estimates; the key observation is that the algorithms span several orders of magnitude in computational cost for comparable solution quality.

Table S2: FLOP estimates for each optimization algorithm on the spike protein ($L = 1,273$).

Algorithm	Per-step cost	Steps	Total FLOPs
Potts (10 sweeps)	~ 100 FLOPs/position	$10 \times 1,273$	$\sim 1.3 \times 10^6$
Ising (2×10^4 sweeps)	~ 32 FLOPs/spin	$2 \times 10^4 \times 3,147$	$\sim 2.0 \times 10^9$
GA (tuned)	see below	$200 \times 1,000$	$\sim 6 \times 10^9$

Potts model ($\sim 1.3 \times 10^6$ FLOPs). Each of the 10 Gibbs sweeps updates all $L = 1,273$ positions. Per position, the computation involves: bias lookup ($K = 6$ values), two neighbor interaction lookups (indexing into a $K \times K$ pairwise matrix), softmax normalization over K states ($\sim 3K$ operations for exponentiation and normalization), and a categorical sample ($\sim K$ operations for cumulative sum and comparison). This totals ~ 100 FLOPs per position.

Ising model ($\sim 2.0 \times 10^9$ FLOPs). Each of the 2×10^4 Gibbs sweeps updates all $N = 3,147$ spins. Per spin: computing the effective field from ~ 12 neighbors requires ~ 24 multiply-add operations, plus the bias term (1 operation), a sigmoid evaluation (~ 5 operations for the approximation), and a Bernoulli sample (2 operations). This totals ~ 32 FLOPs per spin.

Genetic algorithm ($\sim 6 \times 10^9$ FLOPs). With population size 200 and 1,000 generations, each generation involves:

- *Scoring* ($\sim 6L$ FLOPs per individual): GC counting (sum over L positions), repeat penalty lookup ($L-1$ table lookups and additions), and rarity score lookup (L table lookups).

- *Crossover* ($\sim 5L$ FLOPs per offspring): parent selection (random index), uniform crossover mask generation and application.
- *Mutation* ($\sim 15L$ FLOPs per offspring): for each position, generate a random number, compare against mutation rate, and when mutating, perform frequency-weighted codon sampling (cumulative probability lookup).
- *Selection and sorting*: ranking the population by score.

The GA requires $\sim 5,000\times$ more FLOPs than the Potts sampler for comparable solution quality.

GPU energy bounds

We estimate GPU energy on an NVIDIA RTX 3090 (350 W TDP, 35.6 TFLOPS FP32 peak) using two methods:

Idealized lower bound. The idealized energy is computed as:

$$E_{\text{ideal}} = \frac{\text{Total FLOPs}}{\text{Peak FLOP/s}} \times \text{TDP} = \frac{\text{Total FLOPs}}{35.6 \times 10^{12}} \times 350 \text{ W} \quad (\text{S24})$$

This is a generous lower bound: no real workload sustains 100% utilization, and the codon optimization workloads are far too small to saturate a modern GPU.

Measured upper bound. The measured energy multiplies the wall-clock execution time of our JIT-compiled JAX implementation by TDP:

$$E_{\text{measured}} = t_{\text{wall}} \times 350 \text{ W} \quad (\text{S25})$$

This is an upper bound since the GPU may not draw full TDP power during execution.

Table S3 summarizes both estimates. The large gap between idealized and measured energy (2–4 orders of magnitude for the Potts model) reflects the fact that these workloads are too small to efficiently utilize GPU hardware: execution time is dominated by kernel launch overhead, memory latency, and pipeline stalls rather than by floating-point computation.

Table S3: GPU energy estimates on an NVIDIA RTX 3090 (350 W TDP, 35.6 TFLOPS FP32 peak). The Ising wall time was estimated by scaling from an RTX 4080 Laptop measurement.

Algorithm	FLOPs	Idealized energy	Wall time	Measured energy
Potts (10 sweeps)	1.3×10^6	13 μJ	0.7 ms	245 mJ
Ising (2×10^4 sweeps)	2.0×10^9	20 mJ	~ 240 ms	~ 84 J
GA (tuned)	6×10^9	58 mJ	60 ms	21 J

Supplementary Note 3: Algorithm details and hyperparameters

This note provides full hyperparameter tables and algorithmic details for reproducibility.

Hyperparameter tables

Tables S4 to S6 list all parameters used for the results reported in the main text.

Table S4: Potts model hyperparameters.

Parameter	Value
Sequence	SARS-CoV-2 spike protein ($L = 1,273$)
Host organism	<i>E. coli</i> K-12
Codon usage weight w_f	0.1
GC content weight w_{GC}	1.0 (standard) / 2×10^4 (hard)
Repeat penalty weight w_R	0.1 (standard) / 0.2 (hard)
Target GC fraction ρ_T	0.5
Number of Potts states $K_{\max}$	6 (padded)
Annealing steps T	10
$\beta_{\min}$	0.3
$\beta_{\max}$	100
Gibbs sweeps per step	1
Total Gibbs sweeps	10
Adaptive GC multiplier η	1.0
Block decomposition	2-color (even/odd positions)
Independent chains	512

Table S5: Ising model hyperparameters.

Parameter	Value
Energy function weights	Same as Potts (Table S4)
Total spins N	3,147
Average spins per position	2.47
Maximum node degree	12
Annealing steps T	10
Sweeps per step	2,000 (standard) / 4,000 (hard)
Total Gibbs sweeps	2×10^4 (standard) / 4×10^4 (hard)
$\beta_{\min}, \beta_{\max}$	2.0, 200 (standard) / 0.5, 150 (hard)
$P_{\min}, P_{\max}$	2.0, 200 (standard) / 4.0, 150 (hard)
Initial constant- P steps n_{const}	5
Adaptive GC multiplier	0.9
Block decomposition	4-color (position parity $\times$ spin-index parity)
Independent chains	512

Table S6: Genetic algorithm hyperparameters.

Parameter	Value
Population size	200
Generations	1,000
Elite survivors	10
Lucky survivors (random)	2
Mutation rate	0.003
Crossover type	Uniform
Initialization	Frequency-weighted random sampling
Independent chains	512

GA parameter sensitivity

The genetic algorithm’s performance is highly sensitive to the mutation rate. The default mutation rate of 0.05 from Fox et al. [2021] yields scores in the range ~ 550 – 600 on the spike protein under the hard parameter setting, and comparably degraded performance under the standard weights. Reducing the mutation rate to 0.003 is required to achieve scores comparable to the Potts and Ising models under both parameter settings.

This sensitivity has a natural interpretation: the uniform crossover operator already provides substantial exploration by recombining partial solutions from different individuals. A high mutation rate is counterproductive because it disrupts good partial solutions faster than selection can preserve them. The optimal mutation rate balances the need to introduce novel codon choices (which crossover alone cannot produce) against the cost of destroying beneficial codon combinations that selection has accumulated.

We identified the optimal parameters through a systematic sweep over mutation rate (0.001–0.1), population size (50–500), and number of elite survivors (5–20), evaluating each configuration over multiple seeds.

Annealing schedules

Both the Potts and Ising models use simulated annealing Kirkpatrick et al. [1983] with a log-spaced inverse temperature schedule:

$$\beta_t = \beta_{\min} \cdot \left(\frac{\beta_{\max}}{\beta_{\min}} \right)^{t/T} \quad (\text{S26})$$

for $t \in \{0, \dots, T\}$, where T is the number of annealing steps. At each step, the host updates β , recomputes the adaptive GC coefficient, and runs a fixed number of Gibbs sweeps.

For the Ising model, the constraint penalty P follows an independent schedule: it stays constant at $P_{\min}$ for the first n_{const} steps, then ramps logarithmically:

$$P_t = \begin{cases} P_{\min} & t \leq n_{\text{const}} \\ P_{\min} \cdot \left(\frac{P_{\max}}{P_{\min}} \right)^{(t-n_{\text{const}})/(T-n_{\text{const}})} & t > n_{\text{const}} \end{cases} \quad (\text{S27})$$

Low P early in annealing permits invalid-state (defect) transitions that provide shortcuts around energy barriers: when a valid intermediate state k has energy exceeding P , the defect route (costing P) is energetically cheaper. High P at the end enforces strict constraint satisfaction.

Spike protein spin count

The spike protein ($L = 1,273$) maps to $N = 3,147$ domain-wall spins, an average of 2.47 spins per amino acid position. This is less than the maximum of $K_{\max} - 1 = 5$ because many amino acids have fewer than 6 synonymous codons (e.g., methionine and tryptophan have only 1 codon each and contribute 0 spins).

Supplementary Note 4: Adaptive GC coefficient

This note gives the full derivation of the adaptive GC coefficient scheme summarised in Section 4.3 of the main text. The scheme is applied identically in the Potts and Ising formulations; the only differences are (i) in the Ising case the linear GC bias is compiled through the domain-wall encoding into Ising biases (Supplementary Note 1), and (ii) the Ising case relies on the GC-sorted codon ordering described at the end of this note.

Why the GC term cannot be implemented natively

The GC content term from the energy function (Eq. (4) of the main text) couples all positions through a single global quadratic constraint:

$$E_{\text{GC}}(\mathbf{c}) = w_{\text{GC}} \left(\frac{1}{3L} \sum_{p=1}^L g(c_p) - \rho_T \right)^2. \quad (\text{S28})$$

Expanding the square yields an *all-to-all* pairwise interaction between the GC counts of every pair of positions:

$$E_{\text{GC}} = \frac{w_{\text{GC}}}{(3L)^2} \sum_{p,q} g(c_p) g(c_q) - \frac{2w_{\text{GC}}\rho_T}{3L} \sum_p g(c_p) + \text{const.} \quad (\text{S29})$$

Representing this interaction directly on a TSU would require $\binom{L}{2}$ dense couplings. Two fatal problems follow:

- **Connectivity.** The near-term TSU target supports only sparse, locally-connected graphs (see Section 4.7 of the main text). Embedding a complete graph on $L = 1,273$ nodes into that fabric is not feasible.
- **Chromatic number.** Even if the dense connectivity were available, a complete graph on L nodes has chromatic number L . Block Gibbs sampling requires as many sequential update phases per sweep as the chromatic number of the interaction graph, so a dense GC term would collapse the parallel block Gibbs scheme to fully sequential updates—destroying the entire advantage of the thermodynamic chip.

We therefore replace the global quadratic term with a *per-chain linear* approximation that is refreshed between annealing steps. This keeps the graph sparse and 2-colorable (Potts chain) or 4-colorable (Ising DWE), while still steering each chain toward the target GC fraction ρ_T .

Linear approximation and update rule

Let $\rho(\mathbf{c}) := \frac{1}{3L} \sum_p g(c_p)$ denote the GC fraction of a configuration, and let λ be a scalar coefficient, maintained *independently per Markov chain*. We fold λ into the Potts unary biases as

$$\tilde{h}_p(k) = h_p(k) + \lambda g(c_p^{(k)}), \quad (\text{S30})$$

where $h_p(k)$ is the original Potts bias (codon-usage rarity plus, in the Ising case, the DWE constraint contribution). Positive λ favours GC-rich codons, negative λ penalises them.

At each annealing step t the host reads the current GC fraction ρ_t of each chain (computed in a few operations from the last observed state) and updates λ via

$$\lambda_{t+1} = \lambda_t - \eta(\rho_t - \rho_T), \quad (\text{S31})$$

where $\eta > 0$ is an adaptation rate (`gc_coeff_adapt_mult` in the code). If the chain has overshoot the target ($\rho_t > \rho_T$), λ is pushed negative so that the next sampling burst penalises further GC incorporation; if it has undershot, λ is pushed positive. The update is entirely host-side between TSU flashes.

Clamping to the quadratic gradient

Equation (S31) on its own is unstable over a long run: λ can drift well past any value that would actually be implied by the quadratic penalty, at which point the linear GC bias dominates the codon-usage and repeat terms and produces pathological solutions. We therefore bound λ_{t+1} at each step by the gradient of the true quadratic GC term, evaluated at the current GC fraction. From Eq. (S28),

$$\lambda_{\text{true}}(\rho_t) := - \left. \frac{\partial E_{\text{GC}}}{\partial (\text{GC count})} \right|_{\rho_t} = - \frac{2w_{\text{GC}}(\rho_t - \rho_T)}{3L}. \quad (\text{S32})$$

This is the coefficient of the linearisation of Eq. (S28) around ρ_t : adding $\lambda_{\text{true}} \cdot g(c_p^{(k)})$ to every Potts bias reproduces the first-order effect of the true quadratic penalty on a codon’s relative weight at ρ_t . Crucially, the sign of λ_{true} is always aligned with the sign of the increment in Eq. (S31): both push λ negative when $\rho_t > \rho_T$ and positive when $\rho_t < \rho_T$. We define

$$\lambda_{t+1}^{\text{unclamped}} := \lambda_t - \eta(\rho_t - \rho_T), \quad (\text{S33})$$

and then clip to the interval between λ_t and $\lambda_{\text{true}}(\rho_t)$:

$$\lambda_{t+1} = \text{clip}\left(\lambda_{t+1}^{\text{unclamped}}, \min(\lambda_t, \lambda_{\text{true}}(\rho_t)), \max(\lambda_t, \lambda_{\text{true}}(\rho_t))\right). \quad (\text{S34})$$

This enforces two simultaneous constraints:

- **Bound by λ_{true} (do not overshoot).** λ_{t+1} never passes $\lambda_{\text{true}}(\rho_t)$. So the linear surrogate is never stronger, at the current operating point, than the quadratic term it is replacing.
- **Monotonic progression (do not back off).** λ_{t+1} also never retreats past λ_t . This preserves the accumulated pressure on the chain: if a string of previous steps has driven λ strongly negative, a single noisy dip in ρ_t does not immediately undo the correction.

The clamp therefore forces λ to move monotonically toward $\lambda_{\text{true}}(\rho_t)$ from whichever side it starts on. The update is implemented in `adapt_gc_coeff` (`codons/problem.py`).

The per-chain nature of λ is essential: early in annealing, different chains land in very different basins and thus have different ρ_t . A single shared λ would push all chains with the same force regardless of where they actually are, defeating the purpose of running many chains in parallel. In our experiments each of the 512 chains therefore carries its own λ , updated independently via Eq. (S34).

Why not just set $\lambda = \lambda_{\text{true}}$?

The derivation of $\lambda_{\text{true}}(\rho_t)$ in Eq. (S32) is tempting to apply directly: at each step, read off ρ_t , set $\lambda_{t+1} = \lambda_{\text{true}}(\rho_t)$, and let the sampler do the rest. This would replace the integrator of Eqs. (S31) and (S34) with an instantaneous “re-fit” of λ to the current GC fraction, and at first glance it should converge at least as fast. It does not. The direct rule is unstable whenever the GC term is strong enough to materially constrain the optimisation, and in that regime collapses into a period-2 limit cycle in which ρ and λ bounce between two widely separated values on every single annealing step.

Empirical failure mode. Figure S1 shows representative trajectories under the hard weight setting ($w_{\text{GC}} = 10^5$, $w_f = 0.1$, $w_R = 0.2$) on the spike protein, $L = 1,273$, 32 chains, 25 annealing steps with 15 Gibbs sweeps per step. Under the direct rule, after a short transient of only ~ 5 steps every chain enters an exact period-2 cycle: $\rho_t \in \{0.273, 0.638\}$ alternates on successive steps, and the corresponding $\lambda_t \in \{+11.91, -7.25\}$. Those two λ values are exactly $\lambda_{\text{true}}(0.273)$ and $\lambda_{\text{true}}(0.638)$, so the cycle is self-consistent: each step the rule prescribes the λ that precisely drives the sampler to the *other* state of the cycle. The sign of $\rho_{t+1} - \rho_t$ flips at every single step (“flip rate” = 1.00), the mean absolute deviation $|\rho - \rho_T|$ over the last third of the run is ≈ 0.19 (vs 0.005 for the adaptive rule), and the mean total energy over the 2-cycle is $\sim 4,000$ (vs ~ 450 for the adaptive rule): optimisation fails outright.

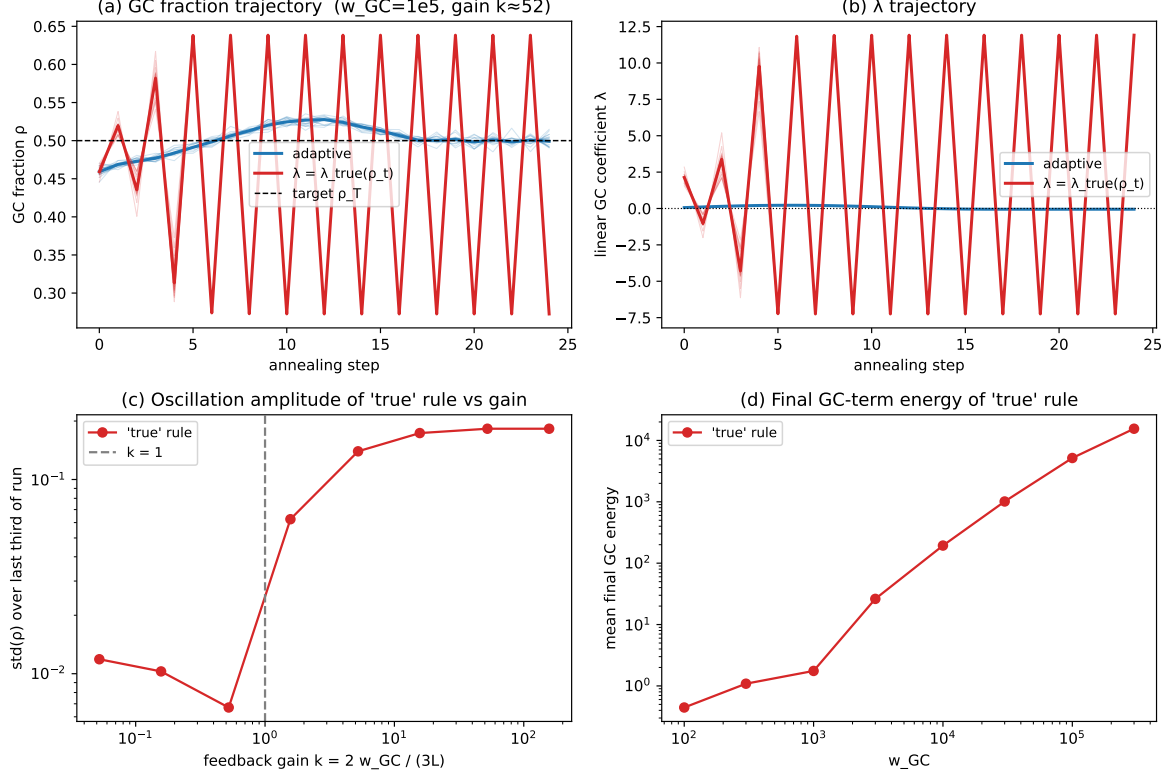


Figure S1: **The direct rule $\lambda = \lambda_{\text{true}}(\rho_t)$ is unstable.** (a,b) GC fraction ρ and linear coefficient λ across annealing steps on the spike protein, $w_{GC} = 10^5$ (hard setting). Light traces: 16 individual chains. Bold: mean over chains. The adaptive rule (blue) quickly settles to $\rho \approx \rho_T = 0.5$ with $\lambda \approx -0.05$; the direct rule (red) locks into a period-2 limit cycle with ρ alternating between 0.27 and 0.64 and λ alternating between +11.9 and -7.3 every single step. (c,d) Sweep of w_{GC} for the direct rule. Oscillation amplitude (std of ρ across the last third of the run) is noise-level below the predicted threshold $k = 2w_{GC}/(3L) = 1$ (dashed line) and saturates at the sampler’s physical response range (≈ 0.18) well above it. The final GC-term energy grows by orders of magnitude across the same sweep.

Linear stability analysis. The direct rule is a pure proportional feedback controller:

$$\lambda_{t+1} = -k(\rho_t - \rho_T), \quad k := \frac{2w_{GC}}{3L}. \quad (\text{S35})$$

The loop is closed by the sampler: given enough Gibbs sweeps at inverse temperature β_t , the new bias λ_{t+1} drives the chain to a new equilibrium GC fraction ρ_{t+1} . For a single categorical position c_p whose Gibbs distribution is $\propto \exp(\beta(h_p(k) + \lambda g(c_p^{(k)})))$ (THRML’s weight convention), the standard exponential-family identity gives $\partial \langle g(c_p) \rangle / \partial \lambda = \beta \text{Var}_p(g)$, and summing over positions yields the per-chain susceptibility

$$\left. \frac{\partial \rho}{\partial \lambda} \right|_{\lambda=0} = \frac{\beta_t}{3L} \sum_{p=1}^L \text{Var}_p(g(c_p)) =: \chi(\beta_t). \quad (\text{S36})$$

Linearising Eq. (S35) together with the sampler response around ρ_T gives

$$\rho_{t+1} - \rho_T \approx -A(\beta_t)(\rho_t - \rho_T), \quad A(\beta_t) = k \cdot \chi(\beta_t). \quad (\text{S37})$$

The fixed point $\rho = \rho_T$ is stable iff $|A| < 1$ and unstable otherwise; the sign of A is always positive, so as soon as $A > 1$ the iteration produces a growing two-sided overshoot, i.e. a geometrically expanding 2-cycle.

For the codon problem on *E. coli*-weighted tables, the per-position variance of the GC count $\text{Var}_p(g)$ averages to $\mathcal{O}(0.3)$ at moderate β_t , giving $\chi(\beta_t) \sim 0.1 \beta_t$ as an order-of-magnitude estimate (the true χ grows sublinearly at large β_t because the per-position distribution concentrates on the locally-favoured codon and its variance decreases). To leading order,

$$A(\beta_t) \sim 0.1 \beta_t \cdot \frac{2w_{\text{GC}}}{3L}. \quad (\text{S38})$$

The annealing schedule spans $\beta_t \in [0.3, 100]$. At the hard-weight setting $w_{\text{GC}} = 10^5$ ($k \approx 52$), this estimate gives A in the range ≈ 1.6 to ≈ 520 – unstable from the very first step. At the standard Fox weights $w_{\text{GC}} = 1$ ($k \approx 5 \times 10^{-4}$), A never exceeds $\approx 5 \times 10^{-3}$, and the direct rule would in fact work there. The oscillation only arises in regimes where the GC term is heavily enforced – which is precisely the regime in which any GC correction is needed at all, and in particular the hard-weight setting used for the stress tests in the main text.

Threshold verified by sweep. Figure S1c,d sweeps w_{GC} through two-and-a-half orders of magnitude for the direct rule. The transition between the stable and oscillating regimes occurs sharply at $k \approx 1$ (between $w_{\text{GC}} = 10^3$ and 3×10^3), as predicted by Eq. (S37). Above that threshold the amplitude grows until it saturates at ≈ 0.18 : with a bias of magnitude $|\beta\lambda| \gg 1$, the sampler pushes every position to its lowest- or highest-GC codon, and the feasible range of ρ over the codon table is the physical ceiling on the 2-cycle’s amplitude.

Why the adaptive rule is stable. Replacing Eq. (S35) with the integral rule of Eq. (S31) gives a two-dimensional linear map in $(\rho - \rho_T, \lambda)$ whose Jacobian has eigenvalues $\{0, 1 - \eta \chi(\beta_t)\}$. The fixed point is stable whenever $\eta \chi(\beta_t) \in (0, 2)$. Crucially, this closed-loop gain is $\eta \chi$, *independent of* w_{GC} : the integrator decouples the per-step feedback gain from the strength of the quadratic penalty it is emulating. Since $\chi(\beta_t)$ is an $\mathcal{O}(1)$ quantity bounded over the entire annealing schedule – at small β_t it scales linearly with slope ~ 0.1 (the average codon-table GC variance divided by three), at large β_t it decays back toward 0 as each position’s distribution concentrates on the locally-favoured codon – a choice of $\eta = \mathcal{O}(1)$ keeps $\eta \chi$ safely inside $(0, 2)$ for *any* w_{GC} . Our production values ($\eta = 1.0$ for Potts and $\eta = 0.9$ for Ising; Tables S4 and S5) sit comfortably in this range.

The clamp of Eq. (S34) plays a secondary but important role: the integrator, left alone, could accumulate λ past λ_{true} at large β_t . At that point the linear surrogate would be *stronger* than the quadratic term it is replacing, and its competition with the codon-usage and repeat terms would distort the objective. The clamp to $\lambda_{\text{true}}(\rho_t)$ is the natural upper bound: once λ reaches λ_{true} , the linear approximation is already as strong as the true quadratic gradient at the current ρ_t , and further integration would over-penalise GC deviation.

In short, the right way to use λ_{true} is as a *bound*, not as a *target*: the adaptive rule walks monotonically toward λ_{true} from the previous value via small integrator steps, while the direct rule jumps to λ_{true} every step – and, because the sampler’s response gain χ is large enough to make $k\chi > 1$ at interesting weights, those jumps do not damp out but lock into a 2-cycle instead.

Integration with annealing and flash-sample-read

Because λ enters the Potts bias linearly, it is scaled by the current inverse temperature β_t before being compiled into the model. The model biases at step t become

$$\tilde{h}_p^{(t)}(k) = \beta_t \left(h_p(k) + \lambda_t g(c_p^{(k)}) \right). \quad (\text{S39})$$

In the Ising case, $\tilde{h}_p^{(t)}$ is subsequently compiled to Ising biases via domain-wall encoding (Supplementary Note 1, Eqs. (S3) and (S17)). The host algorithm at each annealing step is therefore:

1. Read back the state of every chain from the TSU (or its simulator) and compute ρ_t per chain.
2. Update $\lambda_t \rightarrow \lambda_{t+1}$ via Eq. (S34).
3. Combine the new λ_{t+1} with the scheduled β_{t+1} (and, in the Ising case, P_{t+1}) to produce the biases of Eq. (S39).
4. Flash the new weights to the TSU and run `steps_per_beta` Gibbs sweeps.

The cost on a real TSU is *zero* additional flashes: weights would have to be re-flashed every annealing step anyway in order to update β_t (and P_t in the Ising case), so folding the updated λ_{t+1} into the same flash adds no new operations. The additional host work consists only of one reduction per chain (to compute ρ_t) and a handful of scalar operations.

GC-sorted codon ordering

In the Ising / domain-wall formulation, a single spin flip moves the underlying Potts variable between two *adjacent* indices, $k \leftrightarrow k \pm 1$ (see Section 4.4 of the main text). For the linear GC bias in Eq. (S30) to actually guide the domain wall, the codon indices at each position must be arranged so that adjacent indices differ by as little as possible in GC count. Otherwise, moving the domain wall by one spin would cause discontinuous jumps in GC content, which the linear approximation cannot compensate for smoothly.

We therefore order the codons at each position by ascending GC count, with alphabetical tiebreaking among codons of equal GC count. This is the ordering returned by `SORTED_CODON_TABLE` in `codons/problem.py`, and it is used in both the Potts and Ising formulations (the Potts chain does not require it, but using the same ordering keeps weight computation consistent between the two models). Table S7 shows the resulting ordering for three representative amino acids.

Table S7: GC-sorted codon ordering for representative amino acids. Index 0 is the lowest-GC codon; adjacent indices differ by at most 1 in GC count. The number in parentheses is the GC count of each codon.

Amino acid	#Codons	Idx 0	Idx 1	Idx 2	Idx 3	Idx 4	Idx 5	DW spins
Leucine (L)	6	TTA (0)	CTA (1)	CTT (1)	TTG (1)	CTC (2)	CTG (2)	5
Alanine (A)	4	GCA (2)	GCT (2)	GCC (3)	GCG (3)	–	–	3
Phenylalanine (F)	2	TTT (0)	TTC (1)	–	–	–	–	1

Under this ordering every single spin flip changes the GC content of the sequence by at most 1, and flips between codons of equal GC count are GC-neutral. Two consequences follow. First, the GC landscape seen by the domain wall becomes locally smooth: the term $\lambda \cdot g(c_p^{(k)})$ is monotone (up

to ties) in the Potts index k , so after conversion to Ising biases via first differences (Eq. (S3)) the linear GC coefficient produces a spatially uniform shift along each spin chain rather than a sign-alternating pattern. Second, the adaptive scheme of Eq. (S34) acquires a natural interpretation in spin space: λ is a single scalar knob that uniformly shifts the bias along every domain wall, continuously trading off the cost of moving the wall one step inward against the benefit (or cost) of gaining one unit of GC. An arbitrary codon ordering would scramble this relationship and require a per-codon schedule to achieve the same effect, which is incompatible with a single global λ .

References

- Jesse Berwald, Nicholas Chancellor, and Raouf Dridi. Understanding domain-wall encoding theoretically and experimentally. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 381(2241), December 2022. ISSN 1471-2962. doi: 10.1098/rsta.2021.0410.
- Nicholas Chancellor. Domain wall encoding of discrete variables for quantum annealing and QAOA. *Quantum Science and Technology*, 4(4):045004, August 2019. ISSN 2058-9565. doi: 10.1088/2058-9565/ab33c2.
- Jie Chen, Tobias Stollenwerk, and Nicholas Chancellor. Performance of domain-wall encoding for quantum annealing. *IEEE Transactions on Quantum Engineering*, 2:1–14, 2021. ISSN 2689-1808. doi: 10.1109/TQE.2021.3094280.
- Dillion M. Fox, Kim M. Branson, and Ross C. Walker. mRNA codon optimization with quantum computers. *PLOS ONE*, 16(10):e0259101, October 2021. ISSN 1932-6203. doi: 10.1371/journal.pone.0259101.
- Andraž Jelinčič, Owen Lockwood, Akhil Garlapati, Peter Schillinger, Isaac Chuang, Guillaume Verdon, and Trevor McCourt. An efficient probabilistic hardware architecture for diffusion-like models, 2025. URL <https://arxiv.org/abs/2510.23972>.
- S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, May 1983. ISSN 1095-9203. doi: 10.1126/science.220.4598.671.
- David A. Levin, Yuval Peres, and Elizabeth L. Wilmer. *Markov Chains and Mixing Times*. American Mathematical Society, Providence, RI, 2009. ISBN 978-0-8218-4739-8.
- Andrew Lucas. Ising formulations of many NP problems. *Frontiers in Physics*, 2:5, 2014. ISSN 2296-424X. doi: 10.3389/fphy.2014.00005.
- William Whitehead, Zachary Nelson, Kerem Y. Camsari, and Luke Theogarajan. CMOS-compatible Ising and Potts annealing using single-photon avalanche diodes. *Nature Electronics*, 6(12):1009–1019, November 2023. ISSN 2520-1131. doi: 10.1038/s41928-023-01065-0.